



DATA STRUCTURES

CONTENTS



33. Graph ADT and Graph Traversals

T.V. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of Graphs. In this module we will discuss the Graph ADT and Graph Traversals.

Learning Objectives

The learning objectives of the module are as follows:

- To describe the Graph ADT
- To discuss operations associated with graphs
- To explain in detail the breadth first search and depth first search

33.1 Graph ADTs

The ADT Graph is a graph treated as abstract data type, consisting of

- a set V of items (nodes), and
- a set E of edges, each linking 2 nodes.

In the example shown in Figure 33.1 there are 3 nodes and three edges. The example shows a directed graph.

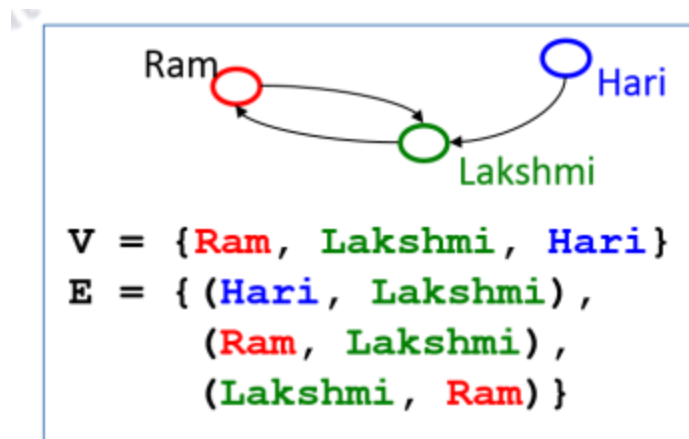


Figure 33.1 An Example Graph

33.1.1 Operations of a typical Graph ADT

The typical operations of a graph are

- Create an empty graph
- Test whether graph is empty.
- Determine the number of vertices in a graph
- Determine the number of edges in a graph
- See whether edge exists between two given vertices -for weighted graphs, return weight value
- Insert vertex in graph whose vertices have distinct values that differ from new vertex's value.
- Insert edge between two given vertices in graph.
- Remove specified vertex from graph and any edges between the vertex and other vertices.

- Remove edge between two vertices in graph.
- Retrieve from graph vertex that contains given value.

33.1.2 Graph Formalism

Graphs are a formalism for representing relationships between entities represented as vertices or nodes

- a graph **G** is represented as

G = (**V**, **E**)

- **V** is a set of vertices
- **E** is a set of edges
- Iterative operations include:
 - iterating over vertices
 - iterating over edges
 - iterating over vertices adjacent to a specific vertex
 - asking whether an edge exists connected two vertices

Iterating over all nodes and edges are defined by

```
forall_nodes(v, G) { ... }
forall_edges(e, G) { ... }
```

which can be used to iterate over all nodes and edges of a graph **G**. The nodes and edges are part of a list **V** and **E**, respectively. We can also define an iterator to iterate over all vertices that are adjacent to a specific vertex.

33.1.3 Example of a Graph ADT

When designing a graph we must decide what needs to be represented explicitly. We can represent vertices explicitly because we want to be able to identify them via a label, and to represent edges implicitly via the vertices they connect. This design decision is reflected in the Graph ADT, which specifies an unweighted graph that could be directed or undirected. The ADT supports adding and deleting vertices and edges, getting the number of vertices and edges in the graph, and getting a list

of the neighbors of a vertex. It is in the addition and deletion of edges that directionality matters.

Thus a typical Graph ADT has vertices and edges . Associated with the Graph ADT are some operations.

- `getDegree( u )` — Returns the degree of vertex u (outdegree of vertex u in directed graph)
- `getAdjacent( u )` — Returns a list of the vertices **adjacent to** vertex u (list of vertices that u points to for a directed graph)
- `isAdjacentTo( u, v )` — Returns TRUE if vertex v is adjacent to vertex u , FALSE otherwise.

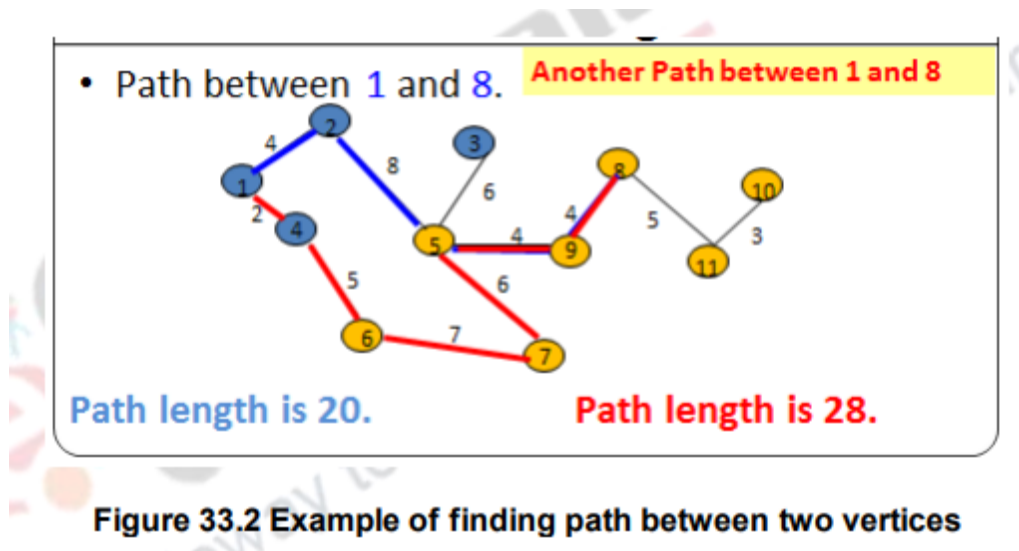
33.2 Graph Operations

Some of the important operations associated with a graph are discussed below. They include graph traversals, path finding, connectedness problems and spanning tree problems.

33.2.1 Traversals and Path Finding

Graph traversal or graph search refers to the process of visiting each vertex in a graph. Traversals are classified based on the order in which the vertices are visited. Given $G=(V,E)$ and vertex v , find all $w \in V$, such that w connects to v . The two types of traversal are Breadth First Search (BFS) and Depth First Search (DFS).

Path Finding is the problem of finding a path between any two vertices as shown in Figure 33.2. There can be more than one path between two vertices.



33.2.2 Connected Graph and Connected Components

A **connected graph**, G , is one in which every vertex in G is reachable from (connected to) every other vertex in G . A connected graph has exactly one component as in Fig 33.3. As discussed in the previous module, a subgraph is a subset of vertices and edges forming a Graph. A maximal subgraph that is connected and where one cannot add vertices and edges from original graph and retain connectedness is called a connected graph.

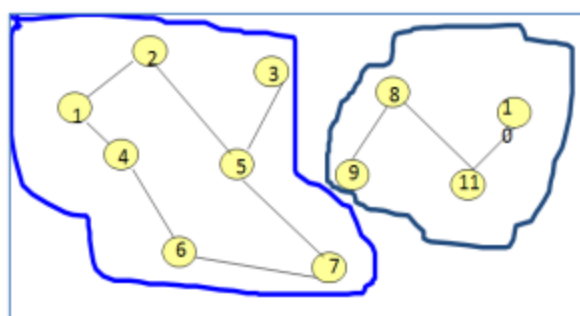


Figure 33.3 Example of Connected Components

33.2.2.1 Communication Network Problems

There are many communication network problems. In a communication network

each edge is a link that can be constructed that is it is a feasible link. One of the connectivity based applications is to find out whether it is possible to communicate between every pair of cities, given a city connection network. Another application is to find all the connected components of a network. Another typical graph operation associated with communication networks is to find the smallest number of feasible links needed to make the resulting connected.

Another problem is to discover the Hamiltonian path if it exists which is an NP complete problem. A graph is Hamiltonian-connected if for every pair of vertices there is a Hamiltonian path between the two vertices. A Hamiltonian cycle, a vertex tour is a cycle that visits each vertex exactly once, except for the vertex that is both the start and end, which is visited twice. Associated applications include the traveling salesperson problem. A variation of Hamiltonian circuit is the weighted graph where the problem is to find the least expensive path.

33.2.2.2 Graphs and Circuits

A circuit is a type of cycle that visits each vertex or each edge exactly once. An **Euler circuit** is one that begins at a specified vertex, passes through each edge exactly once, and ends at the same vertex. If a graph G has an Euler circuit, then all of its vertices must be even vertices. Practical examples involving Euler paths and circuits include the route inspection problem and the problem of the Seven Bridges of Königsberg. The route inspection problem is one where the garbage truck drives down every street in the town without going along the same street twice. The Bridges of Königsberg problem is to determine whether it is possible to walk a route that crosses each bridge in Königsberg (now Kaliningrad) exactly once and return to the starting point.

33.3 Graph-searching Algorithms

The searching a graph involves systematically tracing the edges of a graph to visit the vertices of the graph. This search is used to discover the structure of a graph. There are basically two standard graph-searching algorithms namely Breadth-first Search (BFS) and Depth-first Search (DFS).

33.3.1 Breadth-first search (BFS) traversal

In the Breadth-first search every vertex adjacent to a vertex v is first visited before before visiting any other vertex. This is essentially a first visited, first explored strategy. The idea of breadth first search is to start at a source vertex s and explore the graph outward in all directions level by level, first visiting all vertices that are the neighbours of s (i.e. have distance 1 from s), then vertices that have distance two from s , and so on. In graph terms this translates to a visit pattern that spreads out in waves from the start vertex; the first wave is one edge away from the start vertex; the second wave is two edges away from the start vertex, and so on. In the iterative form the queue data structure is used to store vertices not yet explored.

33.3.1.1 Methodology

Here essentially the frontier between discovered and undiscovered vertices is uniformly expanded across the breadth of the frontier. On each level the search also maintains a frontier. At the start of each level, the frontier associated with each level contains all unvisited neighbours of a vertex which is essentially all vertices in the graph at same level as the vertex. A vertex is “discovered” the first time it is encountered during the search. A vertex is “finished” if all vertices adjacent to it have been discovered. Notice that every root-leaf path in a BFS tree has the fewest number of edges possible (Figure 33.4).

```
For each vertex  $u$  in  $V - \{s\}$ ,  
  color[u] = white;  
  d[u] = infinity;  
  p[u] = NIL  
color[s] = GRAY; d[s] = 0; p[s] = NIL; Q = empty queue ENQUEUE(Q,s),  
while (Q not empty)  
  u = DEQUEUE(Q)  
  for each  $v \in \text{Adj}[u]$   
    if color[v] = WHITE  
      then color[v] = GREY  
      d[v] = d[u] + 1; p[v] = u  
      ENQUEUE(Q, v);  
  color[u] = BLACK;
```

Figure 33.4 Breadth First Algorithm

BFS follows the following steps assuming BFS (Graph G, Vertex v) vertexQueue is a Queue of Vertices

- mark v as visited
 - enqueue v in vertexQueue
 - while vertexQueue is not empty
 - let v be the element removed from the front of vertexQueue
 - for all Vertices w adjacent to v
 - if w has not been visited
 - enqueue w in vertexQueue
 - mark w as visited
- Discover every vertex that is reachable from s

This procedure also produces a BFS tree with root s and including all reachable vertices. It discovers vertices in increasing order of distance from source node and discovers vertices in a series of layers.

The detailed breadth first algorithm is given in Figure 33.4. An example is shown in Figure 33.5. Here the the BFS path is the level by level order shown in the tree. In

the breadth first case the node A is visited first and then its neighbours not yet visited is visited next that is B,F,I. Now B is considered and its neighbours not yet discovered is visited next that is C,G (though F is also its neighbour that node has already been visited). Next F's neighbours D and H are visited, next I's neighbours need to be visited but it has no undiscovered nodes. Next C's neighbour E is visited. Now since all nodes have been discovered the algorithm ends.

33.3.1.2 Analysis

The initialization of the BFS algorithm takes $O(V)$ where V is the number of vertices. Then we have the traversal loop where vertex is enqueued and dequeued at most once, and each operation takes $O(1)$. Therefore the total time for queuing is $O(V)$. The adjacency list of each vertex is scanned at most once. The sum of lengths of all adjacency lists is $O(E)$ where E is the number of edges in the graph. Therefore summing up over all vertices, the total running time of BFS is $O(V+E)$, linear in the size of the adjacency list representation of graph.

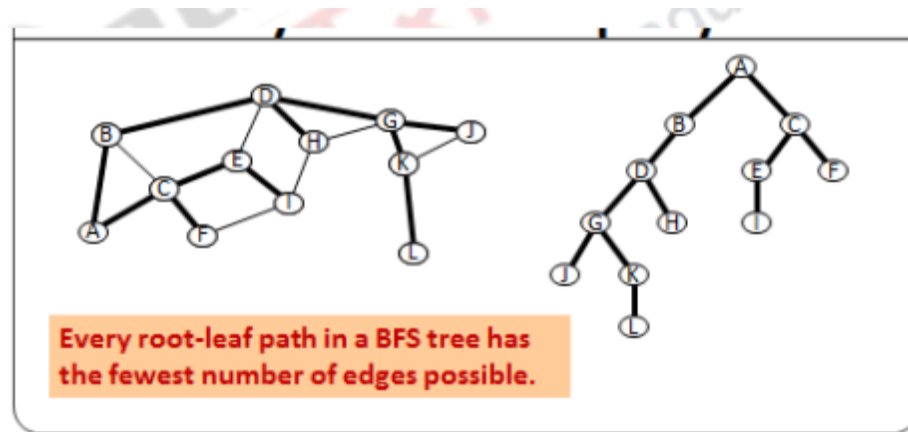


Figure 33.5 Key BFS Tree Property

33.3.1.3 Applications

We can specialize the BFS traversal of a graph G to solve the following problems in $O(n + m)$ time, computing the connected components of G , computing a spanning forest of G , finding a simple cycle in G , or report that G is a forest. Another problem that can be solved is when given two vertices of G , finding a path in G between them with the minimum number of edges, or reporting that no such path exists.

33.3.2 Depth First Search (DFS)

DFS explores edges out of the most recently discovered vertex v . When all edges of v have been explored, backtrack to explore other edges leaving the vertex from which v was discovered (its *predecessor*). The concept is “Search as deep as possible first.” It is continued until all vertices reachable from the original source are discovered. If any undiscovered vertices remain, then one of them is chosen as a new source and search is repeated from that source. Depth-first search (DFS) traversal proceeds along a path from v as deeply into the graph as possible before backing up. The algorithm does not completely specify the order in which it should visit the vertices adjacent to v . It is *essentially* a last visited, first explored strategy. In the iterative form the stack data structure is used to store vertices not yet explored.

33.3.2.1 Methodology

DFS follows the following steps assuming **DFS (Graph G , Vertex v)**

- mark v as visited // don't need to visit a vertex more than once!
- for each Vertex w adjacent to v
- if w has not been visited o DFS(w)

The detailed Depth First algorithm is given in Figure 33.6. An example of traversal is shown in Fig 33.7. Here we start at A and follow one of its neighbours, here B. Now we follow one of B's undiscovered neighbours C, and then C's neighbour D and so on till we reach the node H. Now H has no undiscovered neighbour. We then visit an undiscovered node in the graph, in our case the node I.

DFS(G)

```
for each  $u \in V$  do
  color[u] ← white
  p[u] ← NIL time ← 0
  for each  $u \in V$  do
    if color[u] = white then
      DFS-VISIT(G, u)
      DFS-VISIT(G, u)
      color[u] ← gray
      d[u] ← time ← time + 1
      for each  $v \in \text{Adj}[u]$  do
        if color[v] = white then
          p[v] ← u
          DFS-VISIT(G, v)
          color[v] ← black
          f[u] ← time ← time + 1
```

Figure 33.5 Depth First Algorithm

33.3.2.2 Analysis

The initialization of the BFS algorithm takes $O(V)$ where V is the number of vertices. Then we have the process loop where vertex is stacked and unstacked at most once, and each operation takes $O(1)$. Therefore the total time for stacking is $O(V)$. The adjacency list of each vertex is scanned at most once. The sum of lengths of all adjacency lists is $\sum_{v \in V} |\text{Adj}[v]| = Q(E)$ where E is the number of edges in the graph. Therefore summing up over all vertices, the total running time of BFS is $O(V+E)$, linear in the size of the adjacency list representation of graph.

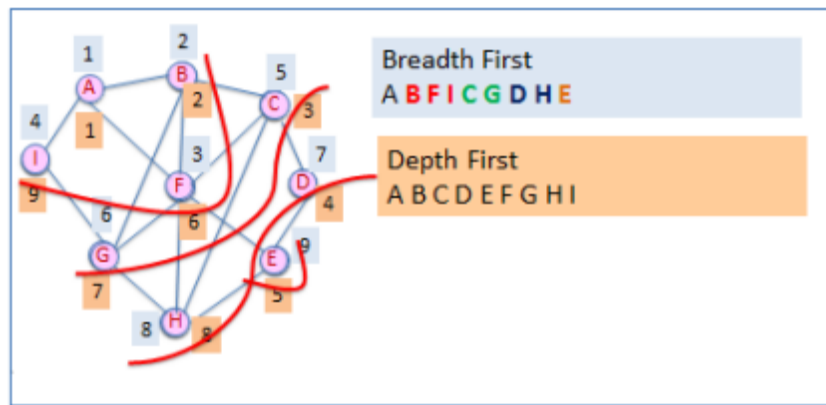


Figure 33.7 Example Traversals

33.3.2.3 DFS: Kinds of Edges

DFS introduces an important distinction among edges in the original graph.

The edges we traverse as we execute a depth-first search can be classified into four edge types as given below (Figure 33.8):

- **Tree edge:** in the depth-first forest is found by exploring (u, v) . The tree edge is the edge (u, v) , which when we traverse the node v is visited for the first time.

The remaining graph edges are classified as:

- **Back edge:** (u, v) , where u is a descendant of v (in the depth-first tree).
- **Forward edge:** (u, v) , where v is a descendant of u , but not a tree edge.
- **Cross edge:** any other edge. Can go between vertices in same depth-first tree or in different depth-first trees.

Tree & back edges are important; most algorithms don't distinguish forward & cross edges

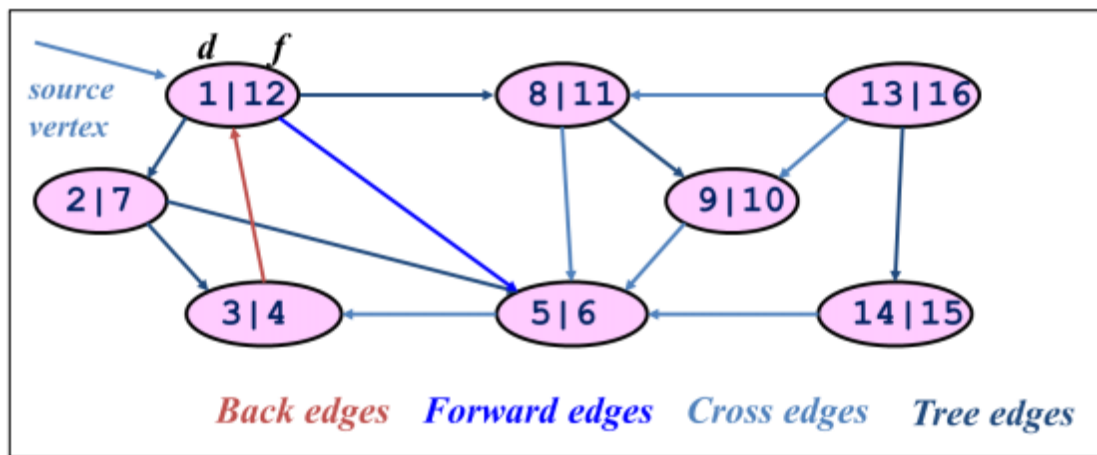


Figure 33.8 Kinds of Edges in Original and Depth First Graph

33.3.2.4 Path Finding

We can specialize the DFS algorithm to find a path between two given vertices u and z . We call $\text{DFS}(G, u)$ with u as the start vertex. We use a stack S to keep track of the path between the start vertex and the current vertex. As soon as destination vertex z is encountered, we return the path as the contents of the stack .

33.3.2.5 Cycle Finding

We can specialize the DFS algorithm to find a simple cycle. We use a stack S to keep track of the path between the start vertex and the current vertex. Here as soon as a back edge (v, w) is encountered, we return the cycle as the portion of the stack from the top to vertex w .

33.4 DFS vs. BFS

Comparison of DFS and BFS is shown in Fig 33.9 and Fig 33.10. As you can see for certain problems like finding spanning forest, connected components, paths and cycles both BFS and DFS can be used. For shortest path BFS algorithm is more efficient. In case of DFS the back edge is in the path of DFS but in BFS it is at same level or next level in the tree of edge discovery.

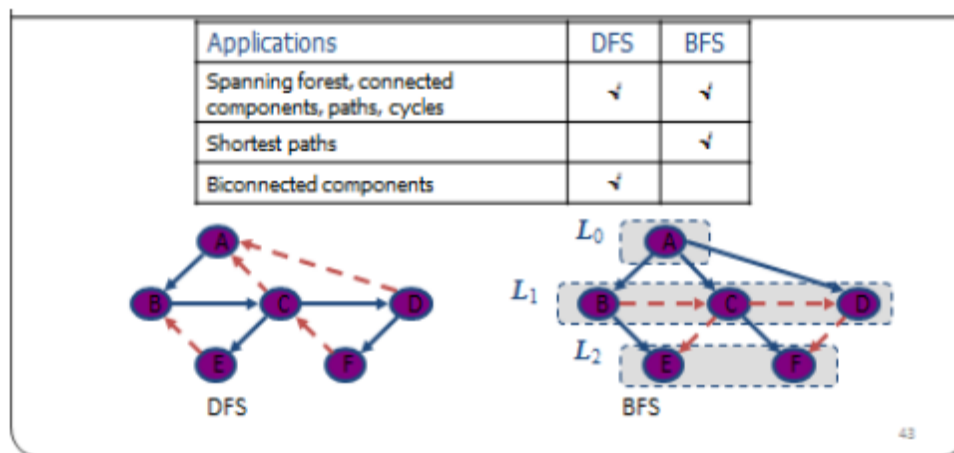


Figure 33.9 DFS vs. BFS Example

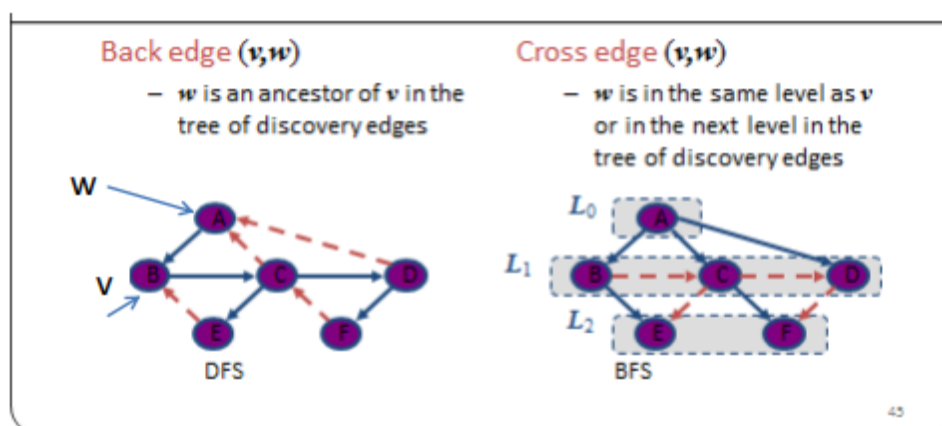


Figure 33.10 Back Edge and Cross Edge

Summary

- Described the Graph ADT
- Discussed operations associated with graphs
- Explained in detail the breadth first search and breadth first search

you can view video on Graph ADT and Graph Traversals



Web Links
http://www.ida.liu.se/opendsa/OpenDSA/Books/TDDI16F15/html/Glossary.html
https://webdocs.cs.ualberta.ca/~holte/T26/graph-traversal.html
https://www8.cs.umu.se/kurser/TDBA77/VT06/algorithms/BOOK/BOOK2/NODE66.HTM
Supporting & Reference Materials
https://en.wikipedia.org/wiki/Book:Introduction_to_Graphs
https://www.cs.purdue.edu/homes/ayg/CS251/slides/chap9b.pdf
http://courses.csail.mit.edu/6.006/fall10/lectures/lec12-graphs2.pdf